

**A REPORT
ON
F.R.I.D.A.Y. - A PERSONAL VOICE ASSISTANT
Chat Bot**

Submitted by

Mr. Reiyan Tahleel(20241CSE1010)

Under the guidance of

**Mr. Sakthi S
Ms. Shaik Salma Begum**

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

AT



PRESIDENCY UNIVERSITY

BENGALURU

AUGUST 2026

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the report of **CSE7000 – Internship** on “**F.R.I.D.A.Y. - A PERSONAL VOICE ASSISTANT CHATBOT**” being submitted by “**Reiyan Tahleel**” bearing roll number “**20241CSE1010**” in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in **Computer Science and Engineering** is a bonafide work carried out under our supervision.

Mr. Sakthi S
Assistant Professor
PSCS
Presidency University

Ms. Shaik Salma Begum
Assistant Professor
PSCS
Presidency University

Dr.R. Balakrishnan
Program Internship
coordinator
PSCS
Presidency University

Dr. Bhuvaneshwari Patil
School level Internship coordinator
PSCS
Presidency University

Dr. Asif Mohamed H.B
Head of the Department
PSCS
Presidency University

Dr. Duraipandian N
Dean – PSCS
Presidency University

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I hereby declare that the work, which is being presented in the report entitled “**F.R.I.D.A.Y. - A Personal Voice Assistant Chatbot**” in partial fulfillment for the award of Degree of **Bachelor of Technology in Computer Science and Engineering** is a record of my own investigations carried under the guidance of **Mr. Sakthi S, Assistant Professor** and **Ms. Shaik Salma Begum, Assistant Professor**, Presidency School of Computer Science and Engineering, Presidency University, Bengaluru.

I have not submitted the matter presented in this report anywhere for the award of any other Degree.

Name: Reiyan Tahleel

Roll No: 20241CSE1010

ACKNOWLEDGEMENTS

First of all, I indebted to the **GOD ALMIGHTY** for giving me an opportunity to excel in our efforts to complete this internship on time.

I express sincere thanks to our respected Dean **Dr. Duraipandian N**, Presidency School of Computer Science and Engineering & Presidency School of Information Science, Presidency University for getting us permission to undergo the internship.

I express heartfelt gratitude to our beloved **Dr. Asif Mohamed H.B**, Head of the Department, Presidency School of Computer Science and Engineering, Presidency University, for rendering timely help in completing this internship successfully.

I am greatly indebted to my reviewers **Mr. Sakthi S Assistant Professor** and **Ms. Shaik Salma Begum, Assistant Professor**, Presidency School of Computer Science and Engineering, Presidency University for their inspirational guidance, and valuable suggestions and for providing a chance to express technical capabilities in every respect for the completion of the internship work.

I would like to convey gratitude and heartfelt thanks to the Internship Coordinator **Dr. Bhuvaneshwari Patil** and Program Internship Coordinator **Dr. R. Balakrishnan**. I also thank my family and friends for the strong support and inspiration they have provided us in bringing out this internship.

Reiyan Tahleel
(20241CSE1010)

ABSTRACT

F.R.I.D.A.Y. is an intelligent, locally hosted personal voice assistant designed to revolutionize natural language interaction for system and process automation.

In the modern era of cloud computing, users frequently rely on public APIs for voice assistance, introducing significant latency and severe data privacy risks. F.R.I.D.A.Y. addresses this challenge by deploying a privacy-first architecture utilizing Local Large Language Models (LLMs).

Powered by Ollama running the llama3.2:3b model locally, the system acts as a highly capable intent parser and tool caller. Users interact with the assistant via an immersive, web-based Arc Reactor Heads-Up Display (HUD). Input is instantly transmitted over a persistent, full-duplex WebSocket connection via a FastAPI backend.

The AI dynamically triggers Python execution scripts to perform system tasks such as web searching, playing media, opening applications, and performing mathematical calculations. Built using Python, FastAPI, WebSockets, HTML/CSS/JS, and the Edge-TTS library, F.R.I.D.A.Y. showcases how open-source LLMs can be utilized natively to create zero-latency, highly capable virtual assistants.

The project proves that complex task automation can be achieved securely on consumer hardware, maintaining conversational context without exposing data to external networks.

The results demonstrate a fully functional, locally hosted AI assistant that offers seamless human-computer interaction, successfully bridging the gap between modern web interfaces and robust system-level automation. The F.R.I.D.A.Y. project proves the viability of utilizing local LLMs to create an engaging, low-latency, and action-oriented digital assistant for daily personal computing tasks.

INTRODUCTION

1.1 Background

The rapid evolution of Artificial Intelligence and Natural Language Processing (NLP) has deeply integrated voice assistants into daily life. However, traditional assistants largely depend on remote cloud processing. This project, F.R.I.D.A.Y. (Female Replacement Intelligent Digital Assistant Youth), is developed to bring that level of intelligence directly to the user's local machine.

1.2 Project Overview

F.R.I.D.A.Y. is a full-stack, real-time interactive voice assistant that processes requests locally using a Local Large Language Model (Ollama). The platform features an interactive web-based Heads-Up Display (HUD) inspired by sci-fi interfaces, communicating with a Python-driven backend via WebSockets. The system acts as a smart agent capable of understanding conversational English and translating it into actionable system commands

PROBLEM STATEMENT

"Modern users and developers lack a secure, customizable, and low-latency virtual assistant platform. Current market solutions rely heavily on cloud-based APIs, resulting in data privacy concerns, noticeable operational latency, and an inability to deeply customize local system automation tasks without steep engineering overhead."

Specific Challenges Identified:

- **Privacy Risks:** Transmitting personal voice data and system queries to third-party cloud servers violates strict privacy preferences.
- **Latency Issues:** Cloud processing requires data round-trips, introducing lag in conversational responses.

OBJECTIVE

The primary objective of this project is to design and develop F.R.I.D.A.Y. (Intelligent Digital Assistant), a localized, voice-interactive artificial intelligence assistant that merges advanced natural language processing with practical system automation. Unlike traditional cloud-based assistants, this project aims to create a highly responsive and privacy-centric system that executes primarily on local hardware.

The specific objectives that guide the development and implementation of this system include:

- **Privacy-First AI:** Implement a virtual assistant that processes requests locally using Llama 3.2, minimizing reliance on cloud APIs.
- **Task Automation:** Automate routine system tasks (e.g. opening applications, playing music on YouTube, math calculations) via natural language.
- **Low-Latency Interaction:** Create a seamless, real-time interface using WebSocket communication between the frontend HUD and backend server.

FEATURES AND MODULES

- **Arc Reactor HUD:** A glowing, responsive frontend UI that tracks the state of the AI (Processing, Listening, Speaking).
- **Full-Duplex Communication Network:** WebSocket integration allows simultaneous data transfer between the browser and backend without HTTP request overhead.
- **Dynamic Tool Calling Framework:** The LLM actively decides which custom Python functions to invoke based on user prompts.
- **Edge-TTS Voice Engine:** Rapidly converts AI text responses into high-quality human speech.

TECHNOLOGY STACK

Category	Technologies
Programming Languages	Python, HTML, CSS, JavaScript
Backend Framework	FastAPI, WebSockets
AI Engine / Brain	Ollama running locally with 'llama3.2:3b'
Voice Synthesis	Edge-TTS Library
Automation Libraries	'pywhatkit ', 'duckduckgo_search ', 'Os'
Development Env.	Antigravity IDE

SYSTEM ARCHITECTURE AND WORKFLOW

7.1 Phase 1: Input Processing

The user interacts through the Arc Reactor HUD web interface. Text input is instantly transmitted to the backend server over a persistent, full-duplex WebSocket connection.

7.2 Phase 2: Natural Language Understanding

The Python backend passes the user's input to the locally hosted Ollama model. The LLM acts as the central brain, analyzing the intent and determining if a specific "tool" needs to be called based on its system prompt.

7.3 Phase 3: Command Execution (Tool Calling)

Based on the LLM's output, specific Python functions are triggered dynamically. For example, if the user asks to play a song, the LLM calls the play_music tool, which triggers pywhatkit to launch YouTube. If a query requires the internet, duckduckgo_search is called

7.4 Phase 4: Output Generation

The LLM generates a concise text response. `edge_tts` instantly converts this text into natural speech, saving it as an audio buffer. The backend streams this binary audio data back through the WebSocket, where the frontend dynamically updates the HUD and auto-plays the audio

IMPLEMENTATION

The implementation of the system is modularized into three core files to ensure clean architecture and separation of concerns:

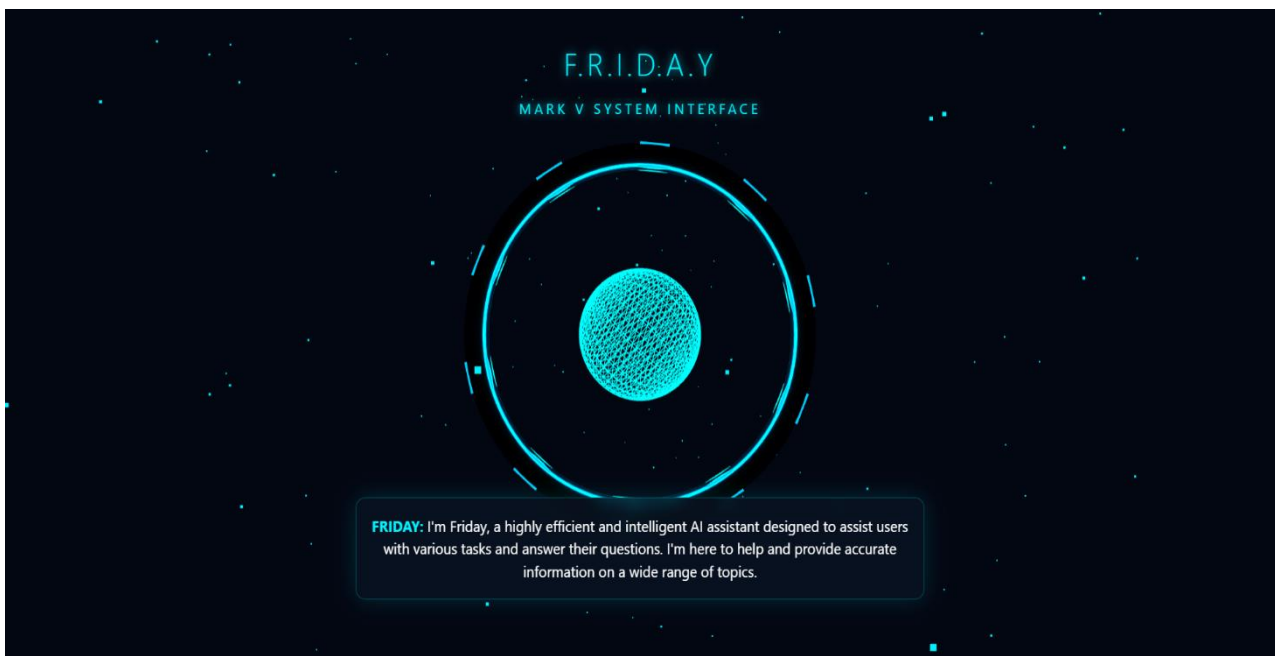
- `index.html` (**Frontend UI**): This file contains the HTML, CSS, and JavaScript required to render the live web playground. It establishes the WebSocket connection to the server, handles user input, visually animates the Arc Reactor based on the AI's state, and natively plays audio blobs streamed from the server without requiring a page refresh.
- `server.py` (**Backend Server**): Built using FastAPI, this file manages the full-duplex WebSocket routes. It maintains the chat history context, interfaces directly with the Ollama API, parses the tool-calling JSON objects returned by the LLM, and handles the asynchronous streaming of text and generated TTS audio back to the client.
- `actions.py` (**Tool Integrations**): This acts as the functional toolbox for the AI. It contains isolated Python definitions for every command the AI can execute. Functions like `open_application()`, `play_music()`, and `calculate_math()` are defined here, ensuring that adding new capabilities to F.R.I.D.A.Y. is as simple as adding a new function to this file

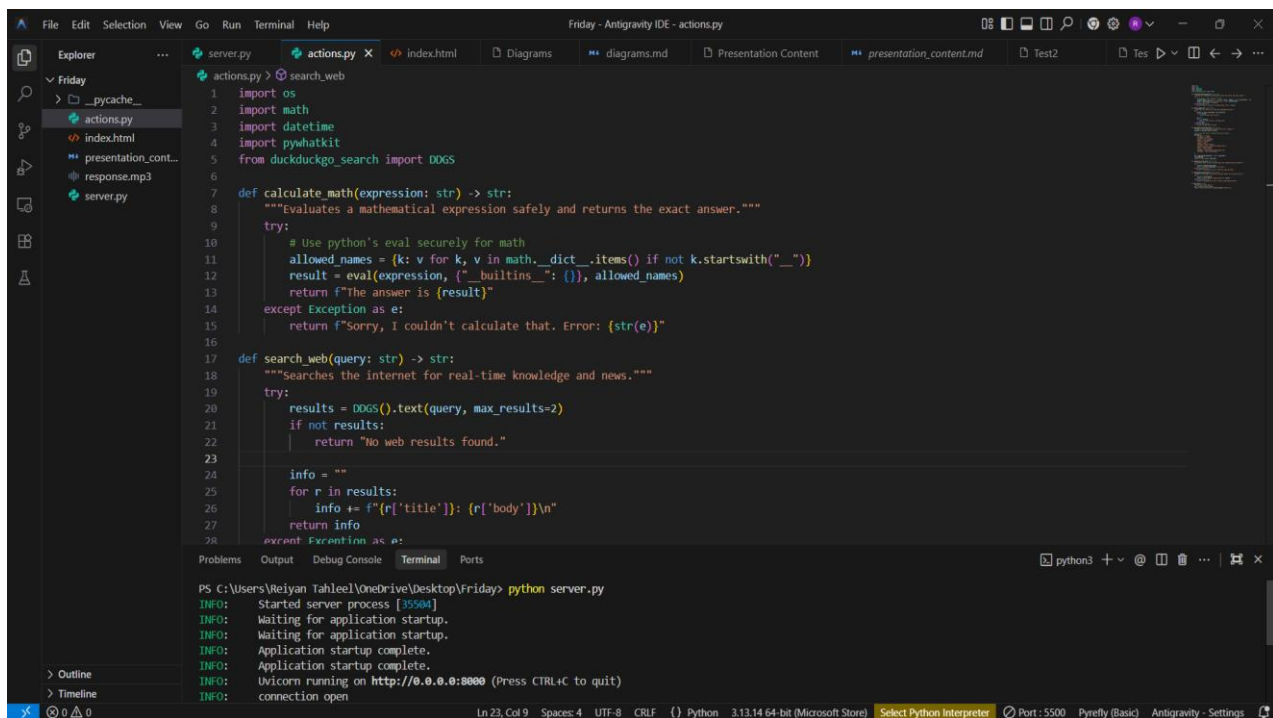
RESULTS

The F.R.I.D.A.Y. AI assistant successfully met its primary objectives. The integration of the llama3.2:3b model via Ollama allowed for accurate natural language processing entirely on local hardware, ensuring data privacy.

The implementation of WebSockets proved highly effective, reducing interaction latency to a minimum compared to traditional REST API polling. F.R.I.D.A.Y. reliably intercepted intents to open system applications, fetch live web search results, and handle mathematical queries dynamically, proving the viability of the actions.py modular tool-calling approach.

SYSTEM SCREENSHOTS





The screenshot displays the Antigravity IDE interface. The Explorer panel on the left shows a project structure with files like `server.py`, `actions.py`, `index.html`, `presentation_content.md`, `response.mp3`, and `server.py`. The main editor window shows the `actions.py` file with the following code:

```
1 import os
2 import math
3 import datetime
4 import pywhatkit
5 from duckduckgo_search import DDGS
6
7 def calculate_math(expression: str) -> str:
8     """Evaluates a mathematical expression safely and returns the exact answer."""
9     try:
10         # Use python's eval securely for math
11         allowed_names = {k: v for k, v in math.__dict__.items() if not k.startswith("__")}
12         result = eval(expression, {"__builtins__": ()}, allowed_names)
13         return f"The answer is {result}"
14     except Exception as e:
15         return f"Sorry, I couldn't calculate that. Error: {str(e)}"
16
17 def search_web(query: str) -> str:
18     """Searches the internet for real-time knowledge and news."""
19     try:
20         results = DDGS().text(query, max_results=2)
21         if not results:
22             return "No web results found."
23
24         info = ""
25         for r in results:
26             info += f"[r['title']]: {r['body']}\n"
27         return info
28     except Exception as e:
29         return f"Error: {str(e)}"
```

The terminal at the bottom shows the command `python server.py` being executed, with output indicating the server process started, application startup completed, and the application is running on `http://0.0.0.0:8000`.

CONCLUSION & FUTURE SCOPE

11.1 Conclusion

The internship and project development provided extensive hands-on experience in building a complex, full-stack AI application. F.R.I.D.A.Y. successfully combined an immersive UI with powerful local LLM-driven automation. Key learnings included the mastery of FastAPI WebSockets, integrating local LLMs with tool-calling capabilities, and designing asynchronous Python architectures.

11.2 Future Enhancements

- Integrating a browser-based microphone API on the frontend for complete hands-free Speech-to-Text (STT) input.
- Expanding the `actions.py` toolkit to interface with IoT devices for smart home automation (e.g., controlling lights or fans).
- Integrating continuous conversation memory to allow for deeply contextual, multi-turn interactions over long periods.

REFERENCES

Ollama and Llama 3: Utilized as the primary Local Large Language Model (LLM) engine, specifically running the 'llama3.2:3b' model for natural language understanding and intent parsing.

FastAPI Framework: Employed for the backend server architecture to manage full-duplex WebSocket routes and interface with the Ollama API.

Edge-TTS Library: Integrated as the voice synthesis engine to rapidly convert AI text responses into natural-sounding human speech.

Python Automation Libraries: Leveraged standard and third-party libraries including 'pywhatkit', 'DuckDuckGo search', 'os', and 'datetime' for executing dynamic tool-calling functions.